

Review: "Educational Insights from Code: Mapping Learning Challenges in Object-Oriented Programming through Code-Based Evidence"

Menolli & Strik (2025) · arXiv:2507.17743

1. Introduction

This paper, authored by André Menolli and Bruno Strik from Brazilian universities, was submitted to SBES 2025 (Brazilian Symposium on Software Engineering). The central question the authors address is: can the quality issues visible in a student's source code reveal what that student actually struggles to understand about object-oriented programming (OOP)? The paper aims to build a conceptual map that connects observable code problems – specifically code smells and violations of the SOLID principles – to documented categories of learning difficulties in OOP. The goal is to give educators a practical tool for diagnosing student understanding through code, rather than just checking whether the code runs correctly.

2. Methodology

The research unfolds in six stages. The authors start from a prior systematic review by Gutiérrez et al. (2022), which identified 14 categories of OOP learning difficulties, and narrow it down to six that are observable in source code. They then apply qualitative content analysis using an inductive open coding approach – analyzing the textual descriptions of each learning difficulty and assigning codes to identify underlying cognitive problems. In parallel, they compile a catalog of code quality indicators from the literature, including implementation-level code smells (Fowler, Brown et al.) and design-level SOLID violations. In the final stage, they deductively map these indicators to the coded learning difficulties. The model is then evaluated by two experienced CS educators who applied it to real student code samples.

3. Results

The study produced a conceptual map organized around six OOP learning challenges: difficulties with classes, methods, the OO paradigm, OO relationships, polymorphism, and encapsulation. For each challenge, the authors identified specific analysis categories and codes, totaling 22 categories and a corresponding set of code indicators. For example, the presence of Long Method and Switch Statements in student code maps to difficulties with method decomposition and polymorphism respectively. The expert evaluation confirmed that the model can trace a meaningful diagnostic path from a code issue to an underlying learning challenge, though some edge cases revealed gaps where the model needs further refinement.

4. Key Insights

Insight 1 – Code quality and conceptual understanding are directly linked. The idea that a God Class or Spaghetti Code in a student's submission is not just bad style but evidence of a specific conceptual gap is genuinely useful. It reframes code review from a grading activity into a diagnostic one. For anyone who will eventually teach, mentor, or do code review, thinking about what a pattern of mistakes reveals

about understanding – rather than just marking it wrong – is a more productive approach.

Insight 2 – SOLID violations as learning signals. The paper shows that violations of principles like Single Responsibility or Open/Closed are not just design problems – they indicate that the student hasn't internalized certain OOP concepts yet. This connection between engineering principles and pedagogical signals is something that's easy to overlook. Knowing it makes SOLID feel less like a checklist and more like a mental model for how programs should be structured, which is a more lasting way to internalize these principles.

Insight 3 – Formative over summative assessment. The paper explicitly targets formative assessment: identifying what the student needs to work on, not just assigning a score. This is relevant in any learning context, including self-directed learning. Looking at one's own code through the lens of code smells and asking what conceptual misunderstanding might be behind a particular pattern is a more honest form of self-evaluation than asking whether the tests pass.

5. Conclusion

This paper makes a practical contribution to programming education by building a structured bridge between software engineering practices and pedagogy. The conceptual map it proposes is grounded in established literature and validated by practitioners, which gives it credibility beyond a purely theoretical exercise. Its main limitation is that the mapping relies on qualitative judgment and doesn't yet cover all possible learning difficulties or all programming languages. The authors themselves propose clear next steps: automated tooling, broader language support, and longitudinal studies to measure the model's actual impact on student learning outcomes. Overall, it is a focused and useful piece of work that raises the right question – not just whether students write correct code, but whether they understand what they are doing.